

# **W12. General-Purpose computing on Graphics Processing Units (GPGPU)**

- Part. 3-

Department of Smart Mobility Engineering  
Inwook, Shim



**인하대학교**  
INHA UNIVERSITY

# Default Setting for colab(2025년 기준)

```
!nvcc -version
```

```
nvcc: NVIDIA (R) Cuda compiler driver  
Copyright (c) 2005-2024 NVIDIA Corporation  
Built on Thu_Jun__6_02:18:23_PDT_2024  
Cuda compilation tools, release 12.5, V12.5.82  
Build cuda_12.5.r12.5/compiler.34385749_0
```

```
!pip install git+https://github.com/andreinechaev/nvcc4jupyter.git
```

```
...
```

```
Successfully built nvcc4jupyter  
Installing collected packages: nvcc4jupyter  
Successfully installed nvcc4jupyter-1.2.1
```

```
%load_ext nvcc4jupyter
```

```
Detected platform "Colab". Running its setup...  
Source files will be saved in "/tmp/tmp5q6zg9uo".
```

# colab test code

```
%%bash
cat > hello.cu << 'EOF'
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void helloCUDA(void)
{
    printf("Hello CUDA from GPU!(Block:%d, Thread:%d)\n",
        blockIdx.x, threadIdx.x);
}

int main(void)
{
    printf("Hello GPU from CPU!\n");
    helloCUDA<<<1,10>>>();

    cudaError_t err = cudaDeviceSynchronize();
    if (err != cudaSuccess) {
        printf("CUDA error after kernel launch: %s\n",
            cudaGetErrorString(err));
        return 1;
    }

    return 0;
}
EOF

nvcc -arch=sm_75 hello.cu -o hello
./hello
```

예상 결과값

```
Hello GPU from CPU!
Hello CUDA from GPU!(Block:0, Thread:0)
Hello CUDA from GPU!(Block:0, Thread:1)
Hello CUDA from GPU!(Block:0, Thread:2)
Hello CUDA from GPU!(Block:0, Thread:3)
Hello CUDA from GPU!(Block:0, Thread:4)
Hello CUDA from GPU!(Block:0, Thread:5)
Hello CUDA from GPU!(Block:0, Thread:6)
Hello CUDA from GPU!(Block:0, Thread:7)
Hello CUDA from GPU!(Block:0, Thread:8)
Hello CUDA from GPU!(Block:0, Thread:9)
```

# Check TPU

```
# @title CheckDim.cu
%%bash
cat > checkdim.cu << 'EOF'
#include <stdio.h>

__global__ void checkIndex(void) {
    printf("threadIdx: (%d, %d, %d) blockIdx: (%d, %d, %d) blockDim: (%d, %d, %d) gridDim: (%d, %d, %d)\n"
        , threadIdx.x, threadIdx.y, threadIdx.z
        , blockIdx.x, blockIdx.y, blockIdx.z
        , blockDim.x, blockDim.y, blockDim.z
        , gridDim.x, gridDim.y, gridDim.z);
}

int main(int argc, char **argv) {
    int nElem = 6;

    dim3 block(3);
    dim3 grid((nElem + block.x - 1) / block.x);

    printf("grid.x %d grid.y %d grid.z %d\n", grid.x, grid.y, grid.z);
    printf("block.x %d block.y %d block.z %d\n", block.x, block.y, block.z);

    checkIndex <<<grid, block>>> ();
    cudaDeviceReset();

    return(0);
}
```

EOF

```
nvcc -arch=sm_75 checkdim.cu -o checkdim
./checkdim
```

예상 결과값

```
grid.x 2 grid.y 1 grid.z 1
block.x 3 block.y 1 block.z 1
threadIdx:(0, 0, 0) blockIdx:(0, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(1, 0, 0) blockIdx:(0, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(2, 0, 0) blockIdx:(0, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(0, 0, 0) blockIdx:(1, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(1, 0, 0) blockIdx:(1, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(2, 0, 0) blockIdx:(1, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
```

# Vector sum

```
# @title new_vector_sum.cu
%%bash
cat > new_vector_sum.cu << 'EOF'

#include "cuda_runtime.h"
#include "device_launch_parameters.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <iostream>
#include <chrono>

using namespace std;
#define NUM_THREAD 32
#define NUM_DATA 20480

__global__ void vecAdd(int *_a, int *_b, int *_c) {
    //printf("BlockID: %d, thread ID: %d\n",
        blockIdx.x, threadIdx.x);
    int tID = blockIdx.x * blockDim.x + threadIdx.x;
    _c[tID] = _a[tID] + _b[tID];
}

int main(void) {
    int *a, *b, *c, *h_c; // Vectors on the host
    int *d_a, *d_b, *d_c; // Vectors on the device
    int memSize = sizeof(int)*NUM_DATA;

    a = new int[NUM_DATA];
    b = new int[NUM_DATA];
    c = new int[NUM_DATA];
    h_c = new int[NUM_DATA];
```

```
memset(a, 0, memSize);
memset(b, 0, memSize);
memset(c, 0, memSize);
memset(h_c, 0, memSize);

// insert random data
for (int i = 0; i < NUM_DATA; i++) {
    a[i] = rand() % 10;
    b[i] = rand() % 10;
}

for (int i = 0; i < NUM_DATA; i++) {
    h_c[i] = a[i] + b[i];
}

dim3 dimGrid(NUM_DATA / NUM_THREAD, 1, 1);
dim3 dimBlock(NUM_THREAD, 1, 1);

cudaMalloc(&d_a, memSize);
cudaMalloc(&d_b, memSize);
cudaMalloc(&d_c, memSize);

cudaMemcpy(d_a, a, memSize, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, b, memSize, cudaMemcpyHostToDevice);

// <<< BlockNum, ThreadNum >>>
// 1개의 블록은 N개의 스레드로 구성
vecAdd<<<dimGrid, dimBlock>>>>(d_a, d_b, d_c);

cudaDeviceSynchronize();
cudaMemcpy(c, d_c, memSize, cudaMemcpyDeviceToHost);
```

# Vector sum

```
cudaFree(d_a);  
cudaFree(d_b);  
cudaFree(d_c);  
  
delete [] a;  
delete [] b;  
delete [] c;  
delete [] h_c;  
  
return 0;  
}
```

EOF

```
nvcc -arch=sm_75 new_vector_sum.cu -o new_vector_sum  
./new_vector_sum
```

```
20480 elements, memSize = 81920 bytes  
VectorSum on Host: 0.082416 ms  
Data Trans. : Host -> Device: 0.109006 ms  
Computation:0.150748 ms  
Data Trans. : Device -> Host: 0.049081 ms  
Cuda Total: 0.34569 ms
```

# Matrix multiplication on CPU

```
# @title mat_mul.cu
%%bash
cat > mat_mul.cu << 'EOF'

#include <iostream>
#include <chrono>

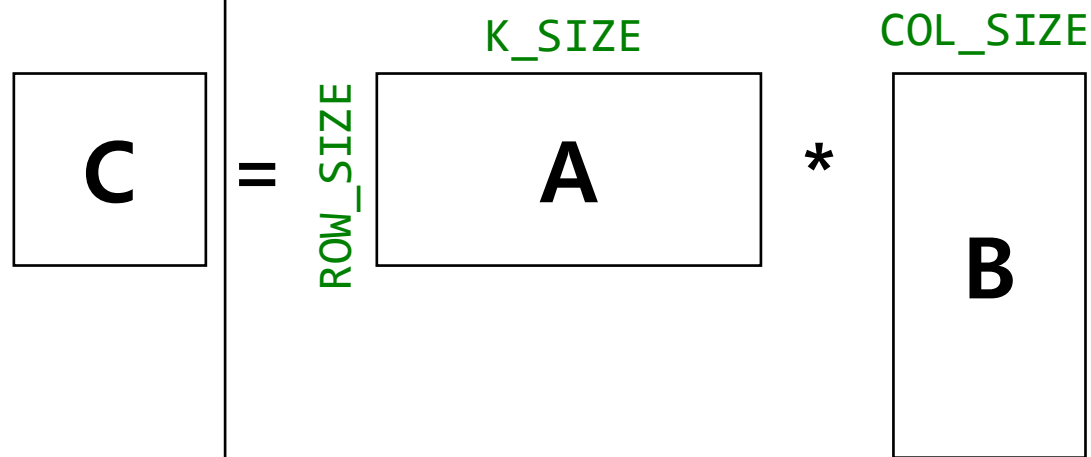
using namespace std;

#define ROW_SIZE (32)
#define K_SIZE (128)
#define COL_SIZE (32)
#define WORK_LOAD (1024)

#define MAT_SIZE_A (ROW_SIZE*K_SIZE)
#define MAT_SIZE_B (K_SIZE*COL_SIZE)
#define MAT_SIZE_C (ROW_SIZE*COL_SIZE)

//input matrix
float A[ROW_SIZE][K_SIZE]; // m * k
float B[K_SIZE][COL_SIZE]; // k * n
//output matrix
float hostC[ROW_SIZE][COL_SIZE]; // host result

#define memsetZero(_P,_type,_size) memset(_P, 0, sizeof(_type)*_size);
```



# Matrix multiplication on CPU

```
void genInputMatrices(void) {
    for (int r = 0; r < ROW_SIZE; r++)
        for (int k = 0; k < K_SIZE; k++)
            A[r][k] = rand() % 100;

    for (int k = 0; k < K_SIZE; k++) {
        for (int c = 0; c < COL_SIZE; c++)
            B[k][c] = rand() % 100;
    }
}

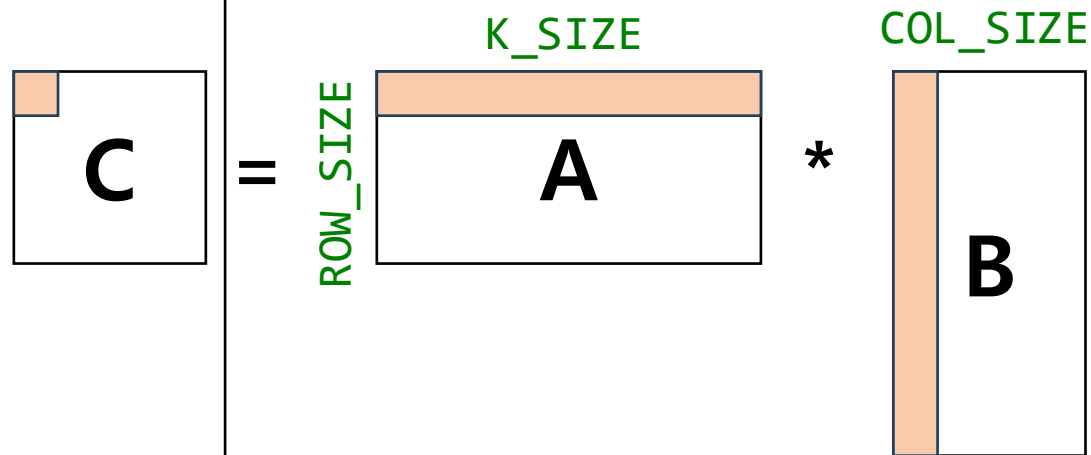
int main(void) {
    genInputMatrices();

    for (int r = 0 ; r < ROW_SIZE ; r++)
        for (int c = 0; c < COL_SIZE; c++)
            for (int k = 0 ; k < K_SIZE; k++)
                for (int i = 0 ; i < WORK_LOAD; i++)
                    hostC[r][c] += A[r][k] * B[k][c];

    return 0;
}

EOF

nvcc -arch=sm_75 mat_mul.cu -o mat_mul
./mat_mul
```



Matrix Multiplication in Host : 503.641 ms



# Matrix multiplication on GPU

```
# @title gpu_mat_mul.cu
%%bash
cat > gpu_mat_mul.cu << 'EOF'

#include <iostream>
#include <chrono>

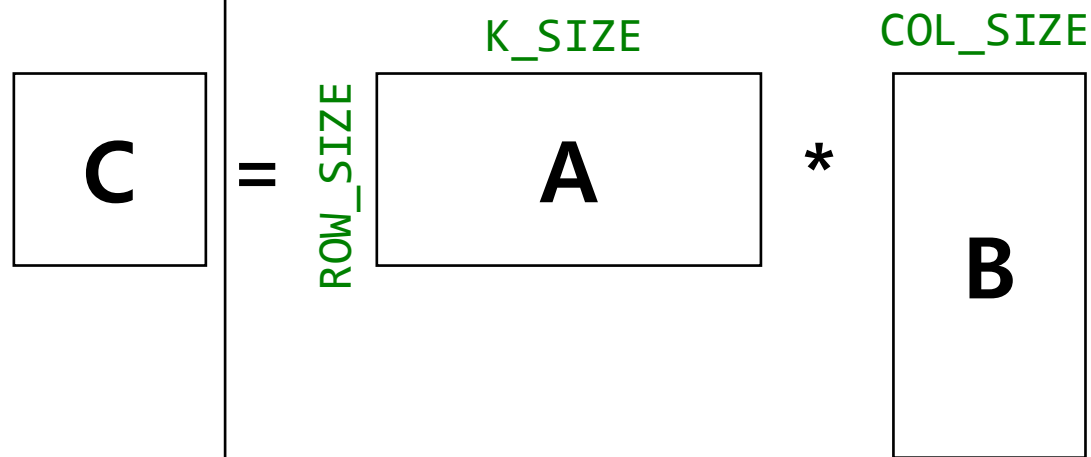
using namespace std;

#define ROW_SIZE (64)
#define K_SIZE (128)
#define COL_SIZE (64)
#define WORK_LOAD (1024)

#define MAT_SIZE_A (ROW_SIZE*K_SIZE)
#define MAT_SIZE_B (K_SIZE*COL_SIZE)
#define MAT_SIZE_C (ROW_SIZE*COL_SIZE)

//input matrix
float A[ROW_SIZE][K_SIZE]; // m * k
float B[K_SIZE][COL_SIZE]; // k * n
//output matrix
float hostC[ROW_SIZE][COL_SIZE]; // host result
float deviceC[ROW_SIZE][COL_SIZE]; // device result

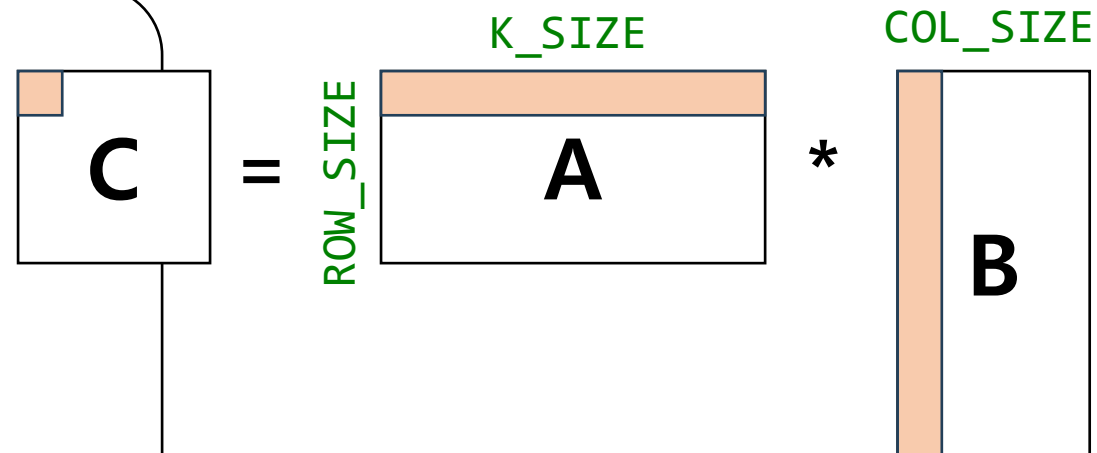
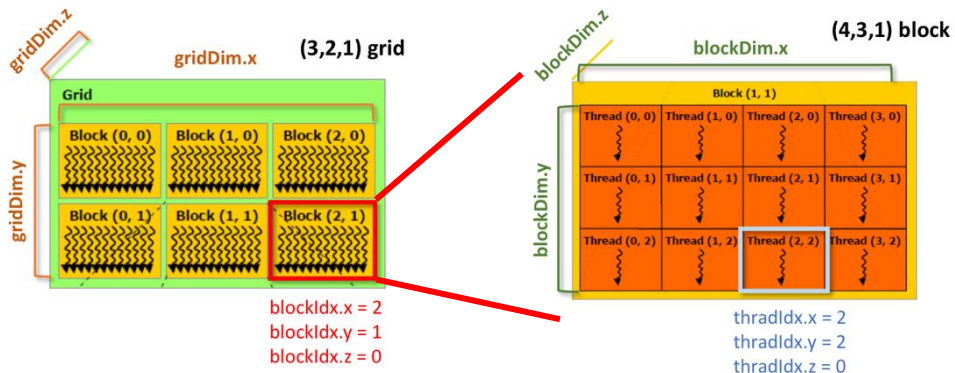
#define memsetZero(_P,_type,_size) memset(_P, 0, sizeof(_type)*_size);
```



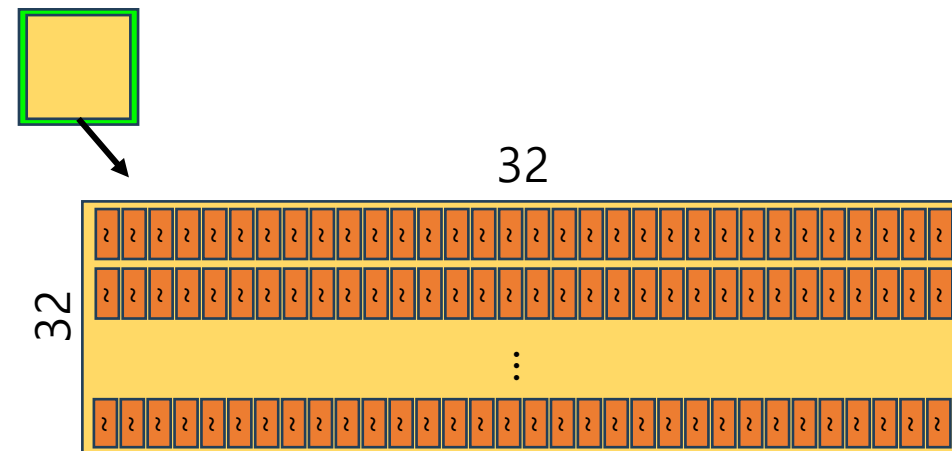
# Matrix multiplication on GPU

```
void genInputMatrices(void) {  
    for (int r = 0; r < ROW_SIZE; r++)  
        for (int k = 0; k < K_SIZE; k++)  
            A[r][k] = rand() % 100;  
  
    for (int k = 0; k < K_SIZE; k++) {  
        for (int c = 0; c < COL_SIZE; c++)  
            B[k][c] = rand() % 100;  
    }  
}
```

```
__global__ void matMul_kernel(float* _A, float* _B, float* _C)  
{  
    int row = threadIdx.y;  
    int col = threadIdx.x;  
    int index = row * blockDim.x + col;  
    _C[index] = 0;  
    for (int k = 0; k < K_SIZE; k++) {  
        for (int i = 0; i < WORK_LOAD; i++) {  
            _C[index] += _A[row * K_SIZE + k] * _B[col + k * COL_SIZE];  
        }  
    }  
}
```



```
dim3 blockDim(COL_SIZE, ROW_SIZE);  
matMul_kernel <<<1, blockDim>>> (...);
```



# Matrix multiplication on GPU

```
int main(void) {
    float *dA, *dB, *dC;
    dA = dB = dC = NULL;
    cudaMalloc(&dA, sizeof(float)*MAT_SIZE_A);
    cudaMalloc(&dB, sizeof(float)*MAT_SIZE_B);
    cudaMalloc(&dC, sizeof(float)*MAT_SIZE_C);

    genInputMatrices();

    for (int r = 0 ; r < ROW_SIZE ; r++)
        for (int c = 0 ; c < COL_SIZE ; c++)
            for (int k = 0 ; k < K_SIZE ; k++)
                for (int i = 0 ; i < WORK_LOAD ; i++)
                    hostC[r][c] += A[r][k] * B[k][c];

    cudaMemcpy(dA, A, sizeof(float)*MAT_SIZE_A, cudaMemcpyHostToDevice);
    cudaMemcpy(dB, B, sizeof(float)*MAT_SIZE_B, cudaMemcpyHostToDevice);

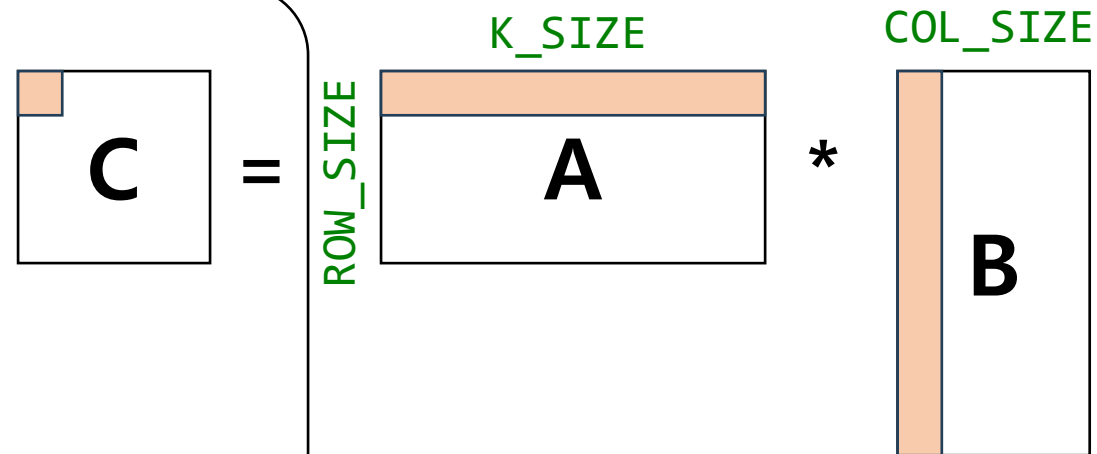
    dim3 blockDim(COL_SIZE, ROW_SIZE);
    matMul_kernel <<<1, blockDim>>> (dA, dB, dC);
    cudaDeviceSynchronize();

    cudaMemcpy(deviceC, dC, sizeof(float)*MAT_SIZE_C, cudaMemcpyDeviceToHost);

    return 0;
}
```

EOF

```
nvcc -arch=sm_75 gpu_mat_mul.cu -o gpu_mat_mul
./gpu_mat_mul
```



```
Matrix Multiplication in Host : 468.091 ms
Copy input matrices : Host -> Device : 0.076553 ms
Computation : 43.6944 ms
Copy output matrices : Device -> Host : 0.033715 ms
Result is correct!
```

# Matrix multiplication on GPU using shared memory

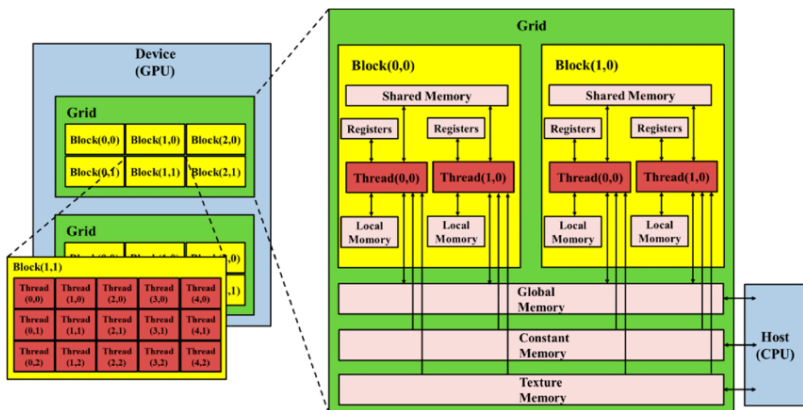
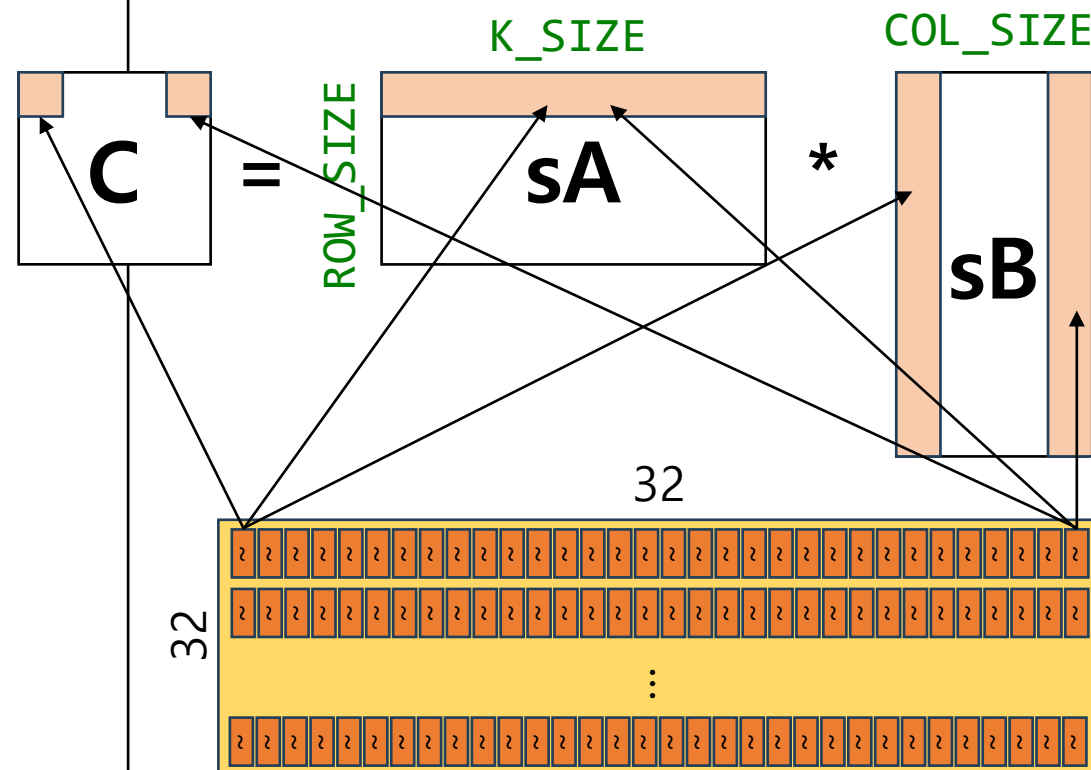
```
__global__ void matMul_kernel_shared(float* _A, float* _B, float* _C)
{
    int row = threadIdx.y;
    int col = threadIdx.x;
    int index = row * blockDim.x + col;

    __shared__ float sA[ROW_SIZE][K_SIZE]; // 32 * 128 * 4 = 16KB
    __shared__ float sB[K_SIZE][COL_SIZE]; // 32 * 128 * 4 = 16KB

    for (int k = 0; k < K_SIZE; k++) {
        sA[row][k] = _A[row * K_SIZE + k];
        sB[k][col] = _B[col + k * COL_SIZE];
    }

    __syncthreads(); // wait until all thread load the matrix

    _C[index] = 0;
    for (int k = 0; k < K_SIZE; k++) {
        for (int i = 0; i < WORK_LOAD; i++) {
            _C[index] += sA[row][k] * sB[k][col];
        }
    }
}
```



Matrix Multiplication in Host : 519.885 ms  
 Copy input matrices : Host -> Device : 0.08458 ms  
 Kernel Computation (Global): 43.6944 ms  
 Kernel Computation (Shared) : 3.88126 ms  
 Copy output matrices : Device -> Host : 0.028304 ms  
 Result is correct!

# CUDA Profiling

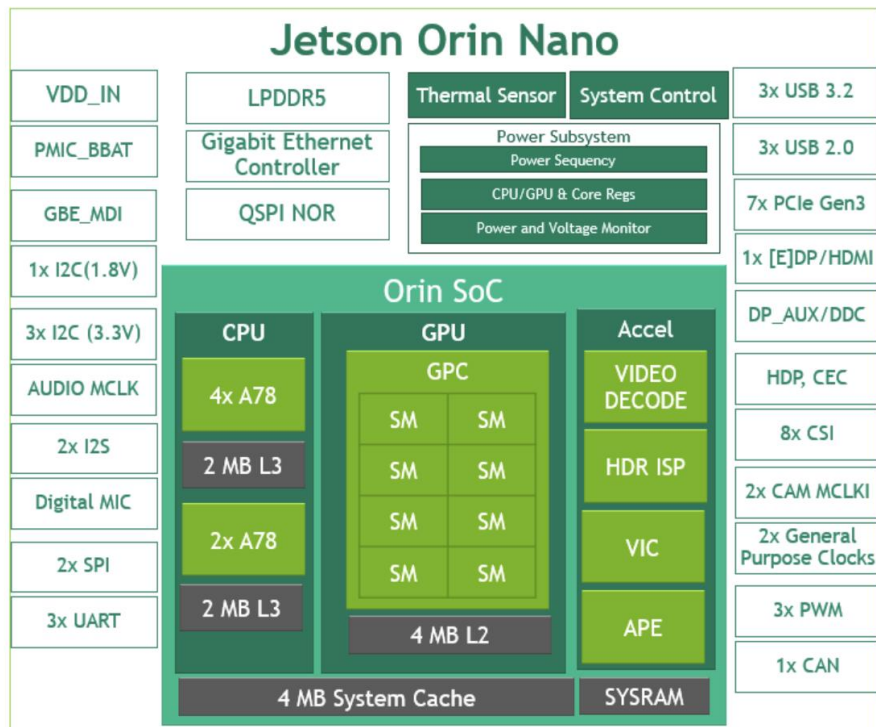


Figure 1. Block diagram of Jetson Orin Nano

\*NVIDIA Orin Architecture from Jetson Orin Nano 8GB, Jetson Orin Nano 4GB has 2 TPCs and 4 SMs.

Jetson nano one SM

Jetson Orin nano 4 SM (4GB) 512 cores

Jetson Orin nano 8 SM (8GB) 1024 cores

Jetson Orin NX 8 SM (8/16GB) 1024 cores

Jetson AGX Orin 16SM (64GB) 2048 cores

Jetson AGX Thor 20SM (128GB) 2560 cores

```
1 !ls
```

```
bin2c          cuda-gdb-python3.8-tui  ncu-ui
compute-sanitizer  cuda-gdb-python3.9-tui  nvcc
crt             cuda-gdbserver        __nvcc_device_query
cudafe++        cu++filt              nvcc.profile
cuda-gdb        cuobjdump            nvdisasm
cuda-gdb-minimal fatbinary             nvlink
cuda-gdb-python3.10-tui gpu_shared_mat_mul     nvprof
cuda-gdb-python3.11-tui gpu_shared_mat_mul.cu  nvprune
cuda-gdb-python3.12-tui ncu                    ptxas
```

```
1 !nvprof ./gpu_shared_mat_mul
```

```
==11931== NVPROF is profiling process 11931, command: ./gpu_shared_mat_mul
```

```
Matrix Multiplication in Host : 495.134 ms
```

```
Copy input matrices : Host -> Device : 0.093172 ms
```

```
Kernel Computation (Shared) : 3.87979 ms
```

```
Copy output matrices : Device -> Host : 0.032407 ms
```

```
Result is correct!
```

```
==11931== Profiling application: ./gpu_shared_mat_mul
```

```
==11931== Profiling result:
```

|                 | Type | Time(%) | Time     | Calls | Avg      | Min      | Max      | Name                                         |
|-----------------|------|---------|----------|-------|----------|----------|----------|----------------------------------------------|
| GPU activities: |      | 99.76%  | 3.6598ms | 1     | 3.6598ms | 3.6598ms | 3.6598ms | matMul_kernel_shared(float*, float*, float*) |
|                 |      | 0.17%   | 6.3680us | 2     | 3.1840us | 2.9120us | 3.4560us | [CUDA memcpy HtoD]                           |
|                 |      | 0.06%   | 2.2720us | 1     | 2.2720us | 2.2720us | 2.2720us | [CUDA memcpy DtoH]                           |
| API calls:      |      | 95.72%  | 92.555ms | 3     | 30.852ms | 2.0340us | 92.549ms | cudaMalloc                                   |
|                 |      | 3.79%   | 3.6629ms | 1     | 3.6629ms | 3.6629ms | 3.6629ms | cudaDeviceSynchronize                        |
|                 |      | 0.22%   | 211.92us | 1     | 211.92us | 211.92us | 211.92us | cudaLaunchKernel                             |
|                 |      | 0.15%   | 143.33us | 114   | 1.2570us | 121ns    | 57.593us | cuDeviceGetAttribute                         |
|                 |      | 0.11%   | 104.09us | 3     | 34.695us | 16.073us | 56.409us | cudaMemcpy                                   |
|                 |      | 0.01%   | 12.086us | 1     | 12.086us | 12.086us | 12.086us | cuDeviceGetName                              |
|                 |      | 0.01%   | 6.0730us | 1     | 6.0730us | 6.0730us | 6.0730us | cuDeviceGetPCIBusId                          |
|                 |      | 0.00%   | 1.1890us | 3     | 396ns    | 144ns    | 846ns    | cuDeviceGetCount                             |
|                 |      | 0.00%   | 888ns    | 2     | 444ns    | 158ns    | 730ns    | cuDeviceGet                                  |
|                 |      | 0.00%   | 382ns    | 1     | 382ns    | 382ns    | 382ns    | cuModuleGetLoadingMode                       |
|                 |      | 0.00%   | 375ns    | 1     | 375ns    | 375ns    | 375ns    | cuDeviceTotalMem                             |
|                 |      | 0.00%   | 225ns    | 1     | 225ns    | 225ns    | 225ns    | cuDeviceGetUuid                              |

# 1M add <<<1,1>>>

```
# @title add_cu_prof.cu
%bash
cat > add_cu_prof.cu << 'EOF'
```

```
#include <iostream>
#include <math.h>
__global__ void add(int n, float *x, float *y) {
    for (int i = 0; i < n; i++)
        y[i] = x[i] + y[i];
}
```

```
int main(void) {
    int N = 1<<20; //1048576
    float *x, *y;
    cudaMallocManaged(&x, N*sizeof(float));
    cudaMallocManaged(&y, N*sizeof(float));

    // initialize x and y arrays on the host
    for (int i = 0; i < N; i++) {
        x[i] = 1.0f;
        y[i] = 2.0f;
    }
```

```
    // Run kernel on 1M elements on the GPU
    add<<<1, 1>>>>(N, x, y);
    cudaDeviceSynchronize();
```

```
    cudaFree(x);
    cudaFree(y);
    return 0;
}
```

```
EOF
```

```
nvcc -arch=sm_75 add_cu_prof.cu -o add_cu_prof
./add_cu_prof
```

- cudaMalloc 은 GPU에만 메모리를 할당
  - cudaMalloc은 메모리 할당 후 CPU와 GPU 간에 데이터를 직접 복사해야 하므로 수동 관리
  - cudaMalloc은 성능 최적화가 중요할 때 사용
- cudaMallocManaged는 CPU와 GPU 모두에서 접근할 수 있는 Unified Memory를 할당
  - 운영체제나 드라이버가 필요에 따라 데이터를 자동으로 이동시켜주므로 사용이 편리
  - cudaMallocManaged는 개발 편의성을 높이고 싶을 때 사용

```
1 !nvprof ./add_cu_prof
```

```
==14540== NVPROF is profiling process 14540, command: ./add_cu_prof
```

```
Max error: 0
```

```
==14540== Profiling application: ./add_cu_prof
```

```
==14540== Profiling result:
```

|                 | Type | Time(%) | Time     | Calls | Avg      | Min      | Max      | Name                     |
|-----------------|------|---------|----------|-------|----------|----------|----------|--------------------------|
| GPU activities: |      | 100.00% | 108.44ms | 1     | 108.44ms | 108.44ms | 108.44ms | add(int, float*, float*) |
| API calls:      |      | 50.05%  | 110.27ms | 2     | 55.137ms | 45.600us | 110.23ms | cudaMallocManaged        |
|                 |      | 49.22%  | 108.45ms | 1     | 108.45ms | 108.45ms | 108.45ms | cudaDeviceSynchronize    |
|                 |      | 0.58%   | 1.2796ms | 2     | 639.80us | 227.76us | 1.0518ms | cudaFree                 |
|                 |      | 0.07%   | 159.25us | 114   | 1.3960us | 110ns    | 70.037us | cuDeviceGetAttribute     |
|                 |      | 0.06%   | 140.80us | 1     | 140.80us | 140.80us | 140.80us | cudaLaunchKernel         |
|                 |      | 0.01%   | 12.302us | 1     | 12.302us | 12.302us | 12.302us | cuDeviceGetName          |
|                 |      | 0.00%   | 6.9640us | 1     | 6.9640us | 6.9640us | 6.9640us | cuDeviceGetPCIBusId      |
|                 |      | 0.00%   | 1.5110us | 3     | 503ns    | 126ns    | 961ns    | cuDeviceGetCount         |
|                 |      | 0.00%   | 1.0020us | 2     | 501ns    | 241ns    | 761ns    | cuDeviceGet              |
|                 |      | 0.00%   | 505ns    | 1     | 505ns    | 505ns    | 505ns    | cuDeviceTotalMem         |
|                 |      | 0.00%   | 383ns    | 1     | 383ns    | 383ns    | 383ns    | cuModuleGetLoadingMode   |
|                 |      | 0.00%   | 244ns    | 1     | 244ns    | 244ns    | 244ns    | cuDeviceGetUuid          |

```
==14540== Unified Memory profiling result:
```

```
Device "Tesla T4 (0)"
```

| Count | Avg Size | Min Size | Max Size | Total Size | Total Time | Name                  |
|-------|----------|----------|----------|------------|------------|-----------------------|
| 48    | 170.67KB | 4.0000KB | 0.9961MB | 8.000000MB | 813.5470us | Host To Device        |
| 24    | 170.67KB | 4.0000KB | 0.9961MB | 4.000000MB | 362.5190us | Device To Host        |
| 12    | -        | -        | -        | -          | 2.497535ms | Gpu page fault groups |

```
Total CPU Page faults: 36
```



# Get device info

```
# @title get_device_info.cu
%%bash
cat > get_device_info.cu << 'EOF'
#include <iostream>
int main() {
    int deviceCount;
    cudaError_t error_id = cudaGetDeviceCount(&deviceCount);

    for (int dev = 0; dev < deviceCount; ++dev) {
        cudaDeviceProp deviceProp;
        std::cout << "\n--- Device " << dev << " Properties ---" << std::endl;
        std::cout << "Name: " << deviceProp.name << std::endl;
        std::cout << "Compute Capability: " << deviceProp.major << "." << deviceProp.minor << std::endl;
        std::cout << "Total Global Memory: " << deviceProp.totalGlobalMem / (1024 * 1024) << " MB" << std::endl;
        std::cout << "Number of Streaming Multiprocessors: " << deviceProp.multiProcessorCount << std::endl;
        std::cout << "Max Threads Per Block: " << deviceProp.maxThreadsPerBlock << std::endl;
        std::cout << "Shared Memory Per Block: " << deviceProp.sharedMemPerBlock / 1024 << " KB" << std::endl;
        std::cout << "Registers Per Block: " << deviceProp.regsPerBlock << std::endl;
        std::cout << "Warp Size: " << deviceProp.warpSize << std::endl;
        std::cout << "Max Threads Per Block: " << deviceProp.maxThreadsPerBlock << std::endl;
        std::cout << "Clock Rate: " << deviceProp.clockRate / 1000 << " MHz" << std::endl;
        std::cout << "Memory Bus Width (bits): " << deviceProp.memoryBusWidth << std::endl;
        std::cout << "Peak Memory Bandwidth (GB/s): "
                    << (2.0*deviceProp.memoryClockRate*(deviceProp.memoryBusWidth/8)/1.0e6) << std::endl;
    }

    return 0;
}
EOF
nvcc -arch=sm_75 get_device_info.cu -o get_device_info
./get_device_info
```

Found 1 CUDA device(s).

--- Device 0 Properties ---

Name: Tesla T4  
Compute Capability: 7.5  
Total Global Memory: 15095 MB  
Number of Streaming Multiprocessors: 40  
Max Threads Per Block: 1024  
Shared Memory Per Block: 48 KB  
Registers Per Block: 65536  
Warp Size: 32  
Max Threads Per Block: 1024  
Clock Rate: 1590 MHz  
Memory Bus Width (bits): 256  
Peak Memory Bandwidth (GB/s): 320.064



# 1M add <<<numBlocks, blockSize>>>

```
# @title add_cu_block_prof.cu
%%bash
cat > add_cu_block_prof.cu << 'EOF'

#include <iostream>
#include <math.h>
// Kernel function to add the elements of two arrays
__global__ void add(int n, float *x, float *y) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = index; i < n; i += stride)
        y[i] = x[i] + y[i];
}

int main(void) {
    int N = 1<<20;
    float *x, *y;

    // Allocate Unified Memory - accessible from CPU or GPU
    cudaMallocManaged(&x, N*sizeof(float));
    cudaMallocManaged(&y, N*sizeof(float));

    // initialize x and y arrays on the host
    for (int i = 0; i < N; i++) {
        x[i] = 1.0f;
        y[i] = 2.0f;
    }

    int blockSize = 1024;
    int numBlocks = (N + blockSize - 1) / blockSize;
    // Run kernel on 1M elements on the GPU
    printf("numBlocks:%d,blockSize:%d\n", numBlocks, blockSize);
    add<<<numBlocks, blockSize>>>(N, x, y);
```

```
// Wait for GPU to finish before accessing on . host
cudaDeviceSynchronize();

// Check for errors (all values should be 3.0f)
float maxError = 0.0f;
for (int i = 0; i < N; i++)
    maxError = fmax(maxError, fabs(y[i]-3.0f));
std::cout << "Max error: " << maxError << std::endl;

// Free memory
cudaFree(x);
cudaFree(y);

return 0;
}

EOF

nvcc -arch=sm_75 add_cu_block_prof.cu -o
add_cu_block_prof
./add_cu_block_prof
```

# How to Optimize Data Transfers in CUDA C/C++

```
1 # @title data_transfer.cu
2 %%bash
3 cat > data_transfer.cu << 'EOF'
4
5 int main()
6 {
7     const unsigned int N = 1048576;
8     const unsigned int bytes = N * sizeof(int);
9     int *h_a = (int*)malloc(bytes);
10    int *d_a;
11    cudaMalloc((int**)&d_a, bytes);
12
13    memset(h_a, 0, bytes);
14    cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);
15    cudaMemcpy(h_a, d_a, bytes, cudaMemcpyDeviceToHost);
16
17    return 0;
18 }
19 EOF
20 nvcc -arch=sm_75 data_transfer.cu -o data_transfer
21 ./data_transfer
```

## GPU activities (실제 GPU에서 일어난 일)

HtoD 723.76us, DtoH 572.79us. 4MB(=1048576 × 4B)

## 대역폭으로 환산하면

- HtoD: 4MB / 723.76 $\mu$ s  $\approx$  5.5 GB/s

- DtoH:  $4\text{MB} / 572.79\mu\text{s} \approx 7.0 \text{ GB/s}$

PCIe 3.0 x16 대역폭은 약 16GB/s

하지만 HtoD / DtoH 모두 0 이 속도의 절반보다 낮음.

>> 원인은 malloc으로 잡은 메모리가 pageable host memory 때문.

```
1 !nvprof ./data_transfer
```

```
==12675== NVPROF is profiling process 12675, command: ./data_transfer
```

```
==12675== Profiling application: ./data_transfer
```

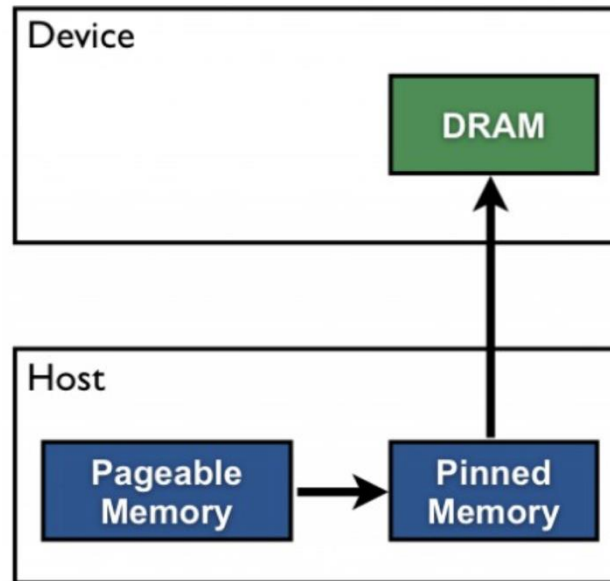
```
==12675== Profiling result:
```

| Type            | Time(%) | Time     | Calls | Avg      | Min      | Max      | Name                   |
|-----------------|---------|----------|-------|----------|----------|----------|------------------------|
| GPU activities: | 55.82%  | 723.76us | 1     | 723.76us | 723.76us | 723.76us | [CUDA memcpy HtoD]     |
|                 | 44.18%  | 572.79us | 1     | 572.79us | 572.79us | 572.79us | [CUDA memcpy DtoH]     |
| API calls:      | 98.98%  | 184.98ms | 1     | 184.98ms | 184.98ms | 184.98ms | cudaMalloc             |
|                 | 0.93%   | 1.7446ms | 2     | 872.30us | 851.79us | 892.81us | cudaMemcpy             |
|                 | 0.08%   | 145.97us | 114   | 1.2800us | 108ns    | 59.618us | cuDeviceGetAttribute   |
|                 | 0.01%   | 13.010us | 1     | 13.010us | 13.010us | 13.010us | cuDeviceGetName        |
|                 | 0.00%   | 5.2800us | 1     | 5.2800us | 5.2800us | 5.2800us | cuDeviceGetPCIBusId    |
|                 | 0.00%   | 1.8940us | 3     | 631ns    | 156ns    | 1.4380us | cuDeviceGetCount       |
|                 | 0.00%   | 1.1260us | 2     | 563ns    | 260ns    | 866ns    | cuDeviceGet            |
|                 | 0.00%   | 552ns    | 1     | 552ns    | 552ns    | 552ns    | cuModuleGetLoadingMode |
|                 | 0.00%   | 545ns    | 1     | 545ns    | 545ns    | 545ns    | cuDeviceTotalMem       |
|                 | 0.00%   | 325ns    | 1     | 325ns    | 325ns    | 325ns    | cuDeviceGetUuid        |

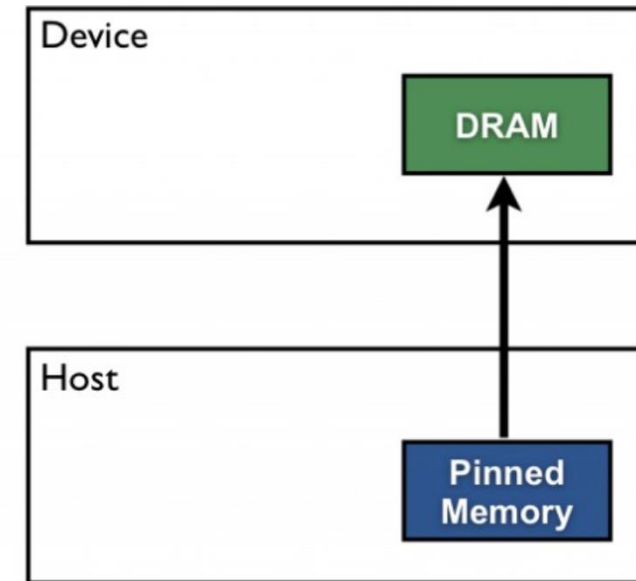
# How to Optimize Data Transfers in CUDA C/C++

- Host (CPU) data allocations are pageable by default.
- The GPU cannot access data directly from pageable host memory.
- When a data transfer from pageable host memory to device memory is invoked, the CUDA driver must first allocate a temporary page-locked, or **"pinned", host array**, copy the host data to the pinned array, and then transfer the data from the pinned array to device memory.
- Pinned memory is used as a staging area for transfers from the device to the host.
- We can directly allocate our host arrays in pinned memory using [cudaMallocHost\(\)](#) or [cudaHostAlloc\(\)](#), and deallocate it with [cudaFreeHost\(\)](#).
- It is possible for pinned memory allocation to fail, so you should always check for errors.

## ***Pageable Data Transfer***



## ***Pinned Data Transfer***



# How to Optimize Data Transfers in CUDA C/C++

```
# @title pinned_data_transfer.cu
%%bash
cat > pinned_data_transfer.cu << 'EOF'

#include <stdio.h>
#include <assert.h>

inline cudaError_t checkCuda(cudaError_t result) {
    if (result != cudaSuccess) {
        fprintf(stderr, "CUDA Runtime Error: %s\n",
            cudaGetErrorString(result));
        assert(result == cudaSuccess);
    }
    return result;
}

void profileCopies(float *h_a, float *h_b, loat *d,
    unsigned int n, char *desc) {
    // 다음 슬라이드에 ...
}

int main() {
    // 다다음 슬라이드에 ...
}

EOF
nvcc -arch=sm_75 pinned_data_transfer.cu -o
pinned_data_transfer
./pinned_data_transfer
```

Device: Tesla T4  
Transfer size (MB): 16

## Pageable transfers

Host to Device bandwidth (GB/s): 4.190516

Device to Host bandwidth (GB/s): 4.289216

## Pinned transfers

Host to Device bandwidth (GB/s): 12.174907

Device to Host bandwidth (GB/s): 13.020613

fin

# How to Optimize Data Transfers in CUDA C/C++

```
void profileCopies(float *h_a, float *h_b, float *d, unsigned int n, char *desc) {
    printf("\n%s transfers\n", desc);
    unsigned int bytes = n * sizeof(float);
    cudaEvent_t startEvent, stopEvent;

    checkCuda( cudaEventCreate(&startEvent) );
    checkCuda( cudaEventCreate(&stopEvent) );

    checkCuda( cudaEventRecord(startEvent, 0) );
    checkCuda( cudaMemcpy(d, h_a, bytes, cudaMemcpyHostToDevice) );
    checkCuda( cudaEventRecord(stopEvent, 0) );
    checkCuda( cudaEventSynchronize(stopEvent) );

    float time;
    checkCuda( cudaEventElapsedTime(&time, startEvent, stopEvent) );
    printf(" Host to Device bandwidth (GB/s): %f\n", bytes * 1e-6 / time);

    checkCuda( cudaEventRecord(startEvent, 0) );
    checkCuda( cudaMemcpy(h_b, d, bytes, cudaMemcpyDeviceToHost) );
    checkCuda( cudaEventRecord(stopEvent, 0) );
    checkCuda( cudaEventSynchronize(stopEvent) );

    checkCuda( cudaEventElapsedTime(&time, startEvent, stopEvent) );
    printf(" Device to Host bandwidth (GB/s): %f\n", bytes * 1e-6 / time);

    for (int i = 0; i < n; ++i)
        if (h_a[i] != h_b[i]) {
            printf("*** %s transfers failed ***\n", desc);
            break;
        }
    // clean up events
    checkCuda( cudaEventDestroy(startEvent) );
    checkCuda( cudaEventDestroy(stopEvent) );
}
```

# How to Optimize Data Transfers in CUDA C/C++

```
int main() {
    unsigned int nElements = 4*1024*1024;
    const unsigned int bytes = nElements * sizeof(float);
    float *h_aPageable, *h_bPageable, *h_aPinned, *h_bPinned; // host arrays
    float *d_a; // device array

    // allocate and initialize
    h_aPageable = (float*)malloc(bytes); // host pageable
    h_bPageable = (float*)malloc(bytes); // host pageable
    checkCuda( cudaMallocHost((void**)&h_aPinned, bytes) ); // host pinned
    checkCuda( cudaMallocHost((void**)&h_bPinned, bytes) ); // host pinned
    checkCuda( cudaMalloc((void**)&d_a, bytes) ); // device

    for (int i = 0; i < nElements; ++i) h_aPageable[i] = i;
    memcpy(h_aPinned, h_aPageable, bytes);
    memset(h_bPageable, 0, bytes);
    memset(h_bPinned, 0, bytes);

    // output device info and transfer size
    cudaDeviceProp prop;
    checkCuda( cudaGetDeviceProperties(&prop, 0) );

    printf("\nDevice: %s\n", prop.name);
    printf("Transfer size (MB): %d\n", bytes / (1024 * 1024));

    // perform copies and report bandwidth
    profileCopies(h_aPageable, h_bPageable, d_a, nElements, (char *)"Pageable");
    profileCopies(h_aPinned, h_bPinned, d_a, nElements, (char *)"Pinned");

    printf("fin");

    // cleanup
    cudaFree(d_a);  cudaFreeHost(h_aPinned);  cudaFreeHost(h_bPinned);  free(h_aPageable);  free(h_bPageable);
    return 0;
}
```

# CUDA stream. Overlap Data Transfers

- How to overlap data transfers with computation on the host
- Computation on the device, and in some cases other data transfers between the host and device.
- Achieving overlap between data transfers and other operations requires the **use of CUDA streams**.
  - A stream in CUDA is a sequence of operations that execute on the device in the order in which they are issued by the host code.
  - While operations within a stream are guaranteed to execute in the prescribed order, operations in different streams can be interleaved and, when possible, they can even run concurrently.

# Default stream

- All device operations (kernels and data transfers) in CUDA run in a stream.
- When no stream is specified, the **default stream** (also called the “**null stream**”) is used.
- The default stream is a synchronizing stream, which means no operation in the default stream will begin until all previously issued operations in any stream on the device have completed.

```
cudaMemcpy(d_a, a, numBytes, cudaMemcpyHostToDevice);  
increment<<<1,N>>>(d_a)  
cudaMemcpy(a, d_a, numBytes, cudaMemcpyDeviceToHost);
```

- All three operations are issued to the same default stream and will execute in the order.
- Since the host-to-device data transfer on the first line is synchronous, the CPU thread will not reach the kernel call on the second line until the host-to-device transfer is complete.
- Once the kernel is issued, the CPU thread moves to the third line, but the transfer on that line cannot begin due to the device-side order of execution.

## Sequential Version





# Default stream

```
cudaMemcpy(d_a, a, numBytes, cudaMemcpyHostToDevice);  
increment<<<1,N>>>(d_a)  
myCpuFunction(b)  
cudaMemcpy(a, d_a, numBytes, cudaMemcpyDeviceToHost);
```

- Simple asynchronous behavior of host and device example.
- We can modify the code to add some independent CPU computation.
- *increment()* kernel is launched on the device the CPU thread executes *myCpuFunction()*, overlapping its execution on the CPU with the kernel execution on the GPU.
- Whether the host function or device kernel completes first doesn't affect the subsequent device-to-host transfer, which will begin **only after** the kernel completes

# Non-default stream

```
cudaStream_t stream1; // declare
cudaError_t result;
result = cudaStreamCreate(&stream1) // create
result = cudaStreamDestroy(stream1) // destroy
```

- a data transfer to a non-default stream

```
result = cudaMemcpyAsync(d_a, a, N, cudaMemcpyHostToDevice, stream1)
```

- It is non-blocking on the host, so control returns the host thread immediately after the transfer function issued. (cudaMemcpy2DAsync() and cudaMemcpy3DAsync())

- issue a kernel to a non-default stream

```
increment<<<1,N,0,stream1>>>(d_a)
```

 3<sup>rd</sup> parameter allocates shared device memory

# Synchronization with streams

- `cudaEventSynchronize(event)` and `cudaEventQuery(event)` act similar to their stream counterparts, except that their result is based on *whether a specified event has been recorded* rather than whether a specified stream is idle.
- You can also synchronize operations within a single stream on a specific event using `cudaStreamWaitEvent(event)` (even if the event is recorded in a different stream, or on a different device!).

# Synchronization with streams

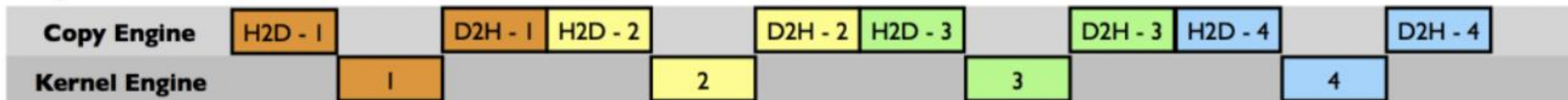
- *cudaDeviceSynchronize()*
  - which blocks the host code until all previously issued operations on the device have completed.
  - In most cases this is overkill, and can really hurt performance due to stalling the entire device and host thread.
- *cudaStreamSynchronize(stream)*
  - block the host thread until all previously issued operations in the specified stream have completed.
- *cudaStreamQuery(stream)*
  - tests whether all operations issued to the specified stream have completed, without blocking host execution.

# Examples

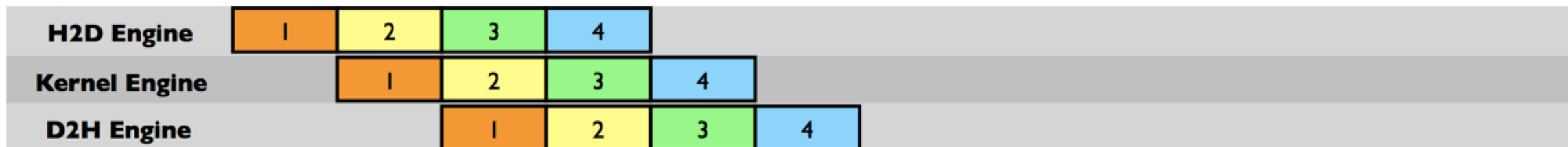
- *The device must be capable of "concurrent copy and execution"*
  - deviceOverlap field of a cudaDeviceProp struct

```
for (int i = 0; i < nStreams; ++i) {  
    int offset = i * streamSize;  
  
    cudaMemcpyAsync(&d_a[offset], &a[offset], streamBytes, cudaMemcpyHostToDevice, stream[i]);  
    kernel<<<streamSize/blockSize, blockSize, 0, stream[i]>>>(d_a, offset);  
    cudaMemcpyAsync(&a[offset], &d_a[offset], streamBytes, cudaMemcpyDeviceToHost, stream[i]);  
}
```

## Asynchronous Version I



## Asynchronous Version I



# Examples

```
for (int i = 0; i < nStreams; ++i) {  
    int offset = i * streamSize;  
    cudaMemcpyAsync(&d_a[offset], &a[offset], streamBytes, cudaMemcpyHostToDevice, stream[i]);  
}  
  
for (int i = 0; i < nStreams; ++i) {  
    int offset = i * streamSize;  
    kernel<<<streamSize/blockSize, blockSize, 0, stream[i]>>>(d_a, offset);  
}  
  
for (int i = 0; i < nStreams; ++i) {  
    int offset = i * streamSize;  
    cudaMemcpyAsync(&a[offset], &d_a[offset], streamBytes, cudaMemcpyDeviceToHost, stream[i]);  
}
```

## Asynchronous Version 2



## Asynchronous Version 2





# Summary

- Multi-thread / Multi-block examples
- simple CUDA profiling nvprof
- pinned data transfer
- Let's see jetson example